

Comparaison de structures de données en C

Tableau dynamique · Liste chaînée · Table de hachage — complexité théorique vs performances mesurées — John WAIA

1. Contexte & complexités théoriques

Trois structures stockent une grande collection d'entiers : un **Dynamic Array** (tableau contigu, doublement de capacité), une **Linked List** simple (tête seule, pas de pointeur de queue) et une **Hash Table** à chaînage (`TABLE_SIZE = 10 007`). Objectif : confronter la complexité asymptotique attendue aux temps réellement mesurés, puis choisir la structure adaptée à deux scénarios d'usage.

Opération	Array	List	Hash Table	Gagnant prévu
get(index)	$O(1)$	$O(n)$	—	Array
find(value)	$O(n)$	$O(n)$	$O(1)$ moy. / $O(n)$ pire cas	Hash Table
insert_front	$O(n)$	$O(1)$	$O(1)$	List
insert_back	$O(1)$ amorti	$O(n)^*$	$O(1)$	Array
remove_front	$O(n)$	$O(1)$	$O(1)$ /élément	List
Parcours complet	$O(n)$ cache-friendly	$O(n)$ cache-hostile	$O(n+TABLE_SIZE)$	Array

* Sans pointeur de queue (structure imposée par l'énoncé), `insert_back` parcourt toute la liste : $O(n)$.

2. Résultats mesurés (n = 1 000 000, MSVC /O2, Windows)

Opération (ms/appel)	Array	List	Hash good	n	Chaîne max (good)	Chaîne max (bad)
get(n/2)	0.000002	2.719783	—	1 000	2	1 000
find(absent)	0.542574	7.776834	0.000346	10 000	7	10 000
insert_front	0.525810	0.000044	—	100 000	24	100 000
insert_back	0.000005	7.816396	—	1 000 000	139	1 000 000
remove_front	0.882027	0.000020	—	<i>hash_bad renvoie toujours 0 → tout tombe dans un seul bucket : la table dégénère en liste chaînée (find à 7.61 ms/appel, quasi identique à la List).</i>		
Parcours (ms total)	0.206	8.345	—			

Chaque structure gagne exactement là où sa complexité le prédit. Array et List sont tous deux $O(n)$ sur `find`/parcours, mais Array reste 4 à 40× plus rapide grâce à la contiguïté mémoire (moins de défauts de cache) : la constante cachée derrière le grand O compte énormément en pratique.

3. Recommandation finale

Situation A — collection quasi statique, 1 000 000 de recherches : Hash Table (`hash_good`). Le coût de construction un peu plus élevé (44 ms vs 8.8 ms pour l'Array) est négligeable face au gain sur `find` (~1500× plus rapide que l'Array, ~22 000× plus rapide que la List), amorti sur un million de recherches.

Situation B — mémoire limitée, seulement 10 recherches : Dynamic Array (trié si possible), pas la Hash Table. Sur 10 recherches, le gain asymptotique ne pèse rien en temps absolu (~5 ms au total), alors que la Hash Table impose un surcoût mémoire fixe permanent (tableau de 10 007 pointeurs, ~80 Ko) injustifié pour si peu d'opérations. Le bon choix dépend du *nombre* d'opérations, pas seulement de leur complexité asymptotique.

Détails complets, méthodologie et sortie brute : voir `README.md` et `results.txt` du dépôt.